

LINUX FOUNDATIONS

# Linux Foundations

Operate the system. Trust the evidence.

Terminal · permissions · SSH · logs · systemd

6-hour workshop

50% theory + demos · 50% hands-on labs

Ubuntu 24.04 browser labs · macOS + Windows



# By the end, you can recover your bearings

## 01 **Navigate**

Move through Linux, manipulate files, and use pipes without losing track of location.

## 02 **Control access**

Reason about users, groups, ownership, modes, sudo, and SSH keys.

## 03 **Diagnose**

Start and stop services, filter logs, identify a cause, and verify the repair.

# Your browser is a window into another Linux system

## BROWSER

**Safari, Edge, or Chrome  
on macOS or Windows**

Shows instructions + terminal

`http://linux-lab.local`

## SESSION

**Ubuntu 24.04  
in an Incus container**

Runs `ls`, `sudo`, `apt`, `systemctl`

`learner@linux-lab:~$`



---

**Browser outside. Linux inside.**

# Practice gets exactly half of the workshop

**180 min**

**Theory ♦ live demos**

- Linux, filesystems, terminal
- Identity, permissions, editor
- SSH, logs, systemd

**180 min**

**Six participant labs**

- Open an isolated lab
- Reset, solve, verify state
- Repair a failed service

Breaks can sit around the blocks; the instructional split stays 50 / 50.

# Linux is the kernel; a usable system is a stack

## Applications

ssh · apt · nano · your services

## User space

shells · libraries · GNU tools · systemd

## Linux kernel

processes · memory · networking · devices

## Hardware / VM / container

CPU · RAM · disks · virtual devices

# The kernel turns hardware into abstractions

**Processes** scheduled programs with isolated address spaces

---

**Memory** virtual memory, pages, caches, and protection

---

**Filesystems** one namespace across disks and virtual data

---

**Network** sockets, routes, interfaces, and packet flow

---

**Devices** drivers expose hardware through common interfaces

# Kernel space and user space fail differently

## KERNEL SPACE

### shared authority

- Drivers and core resource control
- A crash can affect the whole system
- Observed with dmesg and kernel journal fields

## USER SPACE

### isolated processes

- Shells, tools, daemons, applications
- A crash usually affects one process
- Observed with exit status and unit logs

**Locate the failing layer before choosing a tool.**

# A process is a running program with context

## PID

a temporary process identifier

---

## Parent

the process that created it

---

## Identity

effective user and groups used for access checks

---

## Environment

name/value data inherited at process start

---

## Descriptors

open files, sockets, and streams

---



# A distribution is a curated operating agreement

## Same fundamentals

- Kernel interface
- Filesystem concepts
- Users and permissions
- OpenSSH + systemd

## Different choices

- Package format
- Release cadence
- Default configuration
- Support lifecycle

## Choose for context

- Ubuntu / Debian
- RHEL / Rocky / Alma
- Fedora
- Cloud/vendor images

**Today: Ubuntu 24.04 LTS keeps every lab consistent.**

# Distribution families share ideas, not every command

## Debian family

- Ubuntu · Debian
- APT + .deb
- /etc/apt
- Long support

## Red Hat family

- RHEL · Rocky · Alma
- DNF + RPM
- /etc/yum.repos.d
- SELinux defaults

## Other contexts

- Fedora: newer stacks
- Alpine: small images
- Arch: rolling
- Vendor clouds

**Identify the family before copying package or service instructions.**

# Stable and rolling releases trade currency

## STABLE / LTS

### predictability

- Longer support window
- Security fixes with fewer surprises
- Older major versions are common

## ROLLING

### currency

- Frequent package updates
- Fast access to new features
- More integration change over time

**Choose for workload and support needs, not fashion.**

# Record six facts before following external instructions

Distribution

```
cat /etc/os-release
```

---

Kernel

```
uname -r
```

---

Architecture

```
uname -m
```

---

Package tool

```
command -v apt dnf apk
```

---

Init system

```
ps -p 1 -o comm=
```

---

Privilege

```
id; sudo -l
```

---

# Virtual machines isolate kernels; containers share one

## VIRTUAL MACHINE

### hardware boundary

- Guest owns a kernel
- Stronger isolation boundary
- Higher memory and boot cost

## SYSTEM CONTAINER

### OS boundary

- Shares the host kernel
- Own user space and systemd
- Fast cloning and reset

**The browser labs use Incus system containers inside a controller VM.**

# Core operating questions transfer across distributions

## Locate

- Where am I?
- Which host?
- Which user?
- Which file or unit?

## Change

- Predict one state
- Use narrow privilege
- Make one edit
- Capture exit status

## Verify

- Inspect resulting state
- Read relevant logs
- Test intended identity
- Prove behavior

Tools differ; the evidence loop remains stable.

# Most investigations begin at five filesystem anchors

/

one root; every absolute path starts here

---

/etc

host-specific configuration

---

/home

human users' files

---

/var

changing data: logs, caches, queues

---

/usr

installed programs, libraries, shared data

# Filesystem paths express intent, not physical disks

/

the root of the visible namespace

---

Mount

attach another filesystem at a directory

---

Path

a route through directory entries

---

File

a name referring to an inode-like object

---

Metadata

owner, mode, timestamps, size, and type

---



# The hierarchy separates configuration, data, and runtime state

## Configuration

- /etc host settings
- /usr shipped defaults
- Prefer drop-ins
- Track intentional edits

## Persistent data

- /home user files
- /var changing service data
- /srv served content
- /opt optional software

## Runtime state

- /run since boot
- /tmp temporary files
- /proc process views
- /sys device and kernel views

**Directory purpose helps you search before you know an exact filename.**

# /proc and /sys expose live kernel state

## /proc

### process + kernel state

- PID directories describe processes
- cpuinfo and meminfo expose summaries
- Values may change between reads

## /sys

### devices + kernel objects

- Hardware and driver relationships
- Many tunables are writable by root
- Changes can affect the running system

**Read freely; write only with a documented reason.**

# Paths are coordinates; your working directory is the origin

## ABSOLUTE

```
STARTSAT /  
  
/var/log/syslog  
/etc/ssh/sshd_config
```

Works from anywhere because it begins at the root.

## RELATIVE

```
STARTSAT pwd  
  
../archive/report.txt  
./scripts/check.sh
```

Meaning changes when your working directory changes.

**Safety check: pwd → inspect the target → change it → verify the result**

# Path syntax has a small, precise vocabulary

|              |                                                 |
|--------------|-------------------------------------------------|
| <b>/name</b> | absolute: begin at root                         |
| <b>name</b>  | relative: begin at the working directory        |
| <b>.</b>     | the current directory                           |
| <b>..</b>    | the parent directory                            |
| <b>~</b>     | current user's home                             |
| <b>name/</b> | directory path; trailing slash clarifies intent |

# Hidden names are a convention, not protection

## NORMAL LISTING

**ls**

- Omits names beginning with dot
- Keeps routine output quieter
- Permissions are unchanged

## ALL ENTRIES

**ls -la**

- Shows dotfiles and metadata
- Useful for .ssh and configuration
- Still governed by directory access

**A dot changes default visibility; permissions control access.**

# The first character of ls -l identifies object type

|       |                           |
|-------|---------------------------|
| -     | regular file              |
| d     | directory                 |
| l     | symbolic link             |
| c / b | character or block device |
| s     | Unix-domain socket        |
| p     | named pipe                |

# Hard links and symbolic links fail in different ways

## HARD LINK

### another name

- Points to the same inode
- Usually cannot cross filesystems
- Data remains until last link is removed

## SYMBOLIC LINK

### stored path

- Can cross filesystems
- Can point to a directory
- Can become dangling

Inspect with `ls -li` and `readlink` before changing links.

# Globs expand before the command starts

**\***

zero or more characters except leading dot

---

**?**

exactly one character

---

**[ab]**

one character from a set

---

**[0-9]**

one character in a range

---

**Quoted glob**

literal pattern; no pathname expansion

---



# Inspect targets before changing files

1

**Locate**

pwd and realpath

2

**Enumerate**

printf or find the exact targets

3

**Predict**

state the expected names and count

4

**Change**

cp, mv, or rm with narrow scope

5

**Verify**

list, compare, or test the result

**Interactive flags help, but a verified target list is stronger.**

# Live demo: expose a wrong-directory assumption

● ● ● PREDICT → FAIL SAFELY → CORRECT

```
whoami; hostname; pwd
ls /

ls /; cd /var/log; pwd
mkdir -p ~/demo/{in,out}
printf 'ok\nerror: disk\n' > ~/demo/in/app.log

grep "error" ~/demo/in/app.log \
| tee ~/demo/out/errors.txt
```

**The error is evidence: fix the path model, not the permissions.**

# The shell turns text into a process request

● ● ● PROMPT + COMMAND

```
ubuntu@linux-course:~/demo$ grep -i error app.log
```

**grep**

program

**-i**

option

**error**

argument

**app.log**

argument

**Exit status 0 = success · non-zero = a condition or failure**

# Terminal, shell, and command are separate layers

## TERMINAL

### input + display

- Keyboard and screen interface
- Local app or browser ttyd
- Transports control characters

## SHELL

### parser + launcher

- Expands variables and globs
- Connects pipelines and redirections
- Starts programs and reports status

**The command is the program plus the arguments produced by the shell.**

# Quoting decides what the shell expands

**Unquoted** split words, expand variables, expand globs

---

**'single quotes'** preserve every character literally

---

**"double quotes"** expand variables but preserve one argument

---

**backslash** escape the next character

---

**\$()** replace with captured command output

# Shell expansion explains surprising commands

|            |                                                     |
|------------|-----------------------------------------------------|
| Parse      | recognize quotes, operators, and command boundaries |
| Parameters | replace \$name and \${name}                         |
| Commands   | replace \$(...) with output                         |
| Split      | unquoted results may become several words           |
| Glob       | patterns may become many pathnames                  |
| Execute    | run the final program and arguments                 |

# Exit status reports success or failure

**0**

the command reports success

---

**1–125**

program-defined condition or failure

---

**126**

found but not executable

---

**127**

command not found

---

**128 + signal**

terminated by a signal

---

# Control operators and pipes solve different jobs

&& AND ||

## control flow

- Run next only on success
- Or run fallback on failure
- Status determines the branch

| PIPE

## data flow

- Connect stdout to stdin
- Commands run as a pipeline
- Last status is reported by default

**Use control operators for decisions and pipes for streams.**



# Navigate by asking, moving, then confirming

Where am I?

`pwd`

---

What is here?

`ls -la`

---

Move

`cd /var/log`

---

Create

`mkdir -p notes/archive`

---

Inspect

`cat · less · head · tail`

---

Find

`find · grep`

---

# Use local help before trusting a random command

## Quick syntax

- `command --help`
- Short option summary
- Examples may be included
- Best for recall

## Full manual

- `man command`
- `/pattern` to search
- `n` for next result
- `q` to leave

## Shell built-ins

- `help cd`
- `type command`
- `command -V name`
- distinguish aliases

**Local documentation matches the installed version.**

# APT separates repositories, indexes, and installed files

## Repository

- Signed metadata
- Package archives
- Release channels
- Repository trust keys

## Local index

- Refreshed by apt update
- Available versions
- Candidate selection
- Dependency metadata

## Installed state

- Tracked by dpkg
- Files on disk
- Package status
- APT history log

**apt update refreshes knowledge; apt install changes installed state.**

# Use APT with an inspect-change-verify rhythm

Refresh

```
sudo apt update
```

---

Search

```
apt search tree
```

---

Inspect

```
apt show tree
```

---

Install

```
sudo apt install tree
```

---

Verify

```
dpkg -s tree; command -v tree
```

---

Audit

```
less /var/log/apt/history.log
```

---

# Live demo: narrate state, not keystrokes

● ● ● BROWSER → SESSION → PIPELINE

```
whoami; hostname; pwd  
ls /
```

```
ls /; cd /var/log; pwd  
mkdir -p ~/demo/{in,out}  
printf 'ok\nerror: disk\n' > ~/demo/in/app.log
```

```
grep "error" ~/demo/in/app.log \  
| tee ~/demo/out/errors.txt
```

**Predict → run → read output → verify state**

# Pipes connect small tools into an investigation

● ● ● ONE QUESTION, THREE FILTERS

```
journalctl -u ssh --since today | grep -i failed | tail -n 20
```

|

send output to the next command

>

replace a file with output

>>

append output to a file

**Rule: inspect before** ✦ — redirection happens before the command runs.

# Processes start with three standard streams

**stdin**

file descriptor 0: keyboard or pipe input

---

**stdout**

file descriptor 1: normal output

---

**stderr**

file descriptor 2: diagnostics

---

**>**

replace a file with stdout

---

**2>**

replace a file with stderr

---

# Build pipelines as readable evidence stages

1

## Question

state the answer you  
need

2

## Source

choose the smallest  
relevant input

3

## Filter

remove unrelated  
records

4

## Transform

extract, sort, or count

5

## Verify

inspect samples and  
totals

**A long pipeline is maintainable when each stage has one job and can be tested independently.**



# Silence does not prove success

## HIDDEN ERROR

**2>/dev/null**

- Removes diagnostic evidence
- Can turn a real failure into silence
- Acceptable only when failure is expected

## CAPTURED ERROR

**2>error.log**

- Preserves evidence
- Can be reviewed or attached
- Keeps stdout usable as data

**Capture first; suppress only with an explicit reason.**

# Linux authorizes identities, not job titles

USER

**UID 1001**  
**alice**

PRIMARY ♦ SUPPLEMENTARY GROUPS

**alice · courseops · sudo**

IDENTITY CHECKS

```
whoami
id
groups
getent passwd alice
getent group courseops
```

**Processes run as a user.**  
**Files have an owner ♦ group.**  
**Access checks combine both.**

# Names are convenient; numeric IDs drive authorization

|                      |                                           |
|----------------------|-------------------------------------------|
| <b>Username</b>      | human-readable account label              |
| <b>UID</b>           | numeric user identity used by the kernel  |
| <b>Group name</b>    | human-readable collaboration label        |
| <b>GID</b>           | numeric group identity stored on files    |
| <b>Effective IDs</b> | identity currently used for access checks |

# Account databases separate public identity from secrets

## /etc/passwd

- name and UID
- primary GID
- home directory
- login shell

## /etc/group

- group name and GID
- supplementary members
- not every primary member
- readable account data

## /etc/shadow

- password hashes
- aging fields
- root-readable only
- never copy casually

Use getent so local files and network identity sources are queried consistently.

# Existing shells keep old group credentials

## ACCOUNT DATABASE

### updated now

- usermod changes membership
- getent shows the new record
- Future sessions inherit it

## CURRENT SHELL

### old credentials

- Keeps groups from login time
- id may not show the change
- Re-login or newgrp refreshes context

**Always test membership in the process that will do the work.**

# Use sudo for one deliberate action

## Normal shell

least privilege  
clear ownership  
smaller blast radius

✦ sudo

## One privileged command

A terminal window with a dark background and light text. The title bar shows three colored circles (red, yellow, green) and the text "EXPLICIT INTENT". The terminal contains three lines of text: "sudo apt install tree", "sudo systemctl restart ssh", and "sudoedit /etc/ssh/sshd\_config".

```
EXPLICIT INTENT  
sudo apt install tree  
sudo systemctl restart ssh  
sudoedit /etc/ssh/sshd_config
```

**Avoid living in a root shell. Ask: what exact command needs privilege?**

# sudo policy answers who may run what as whom

## Caller

the authenticated invoking user

---

## Target

root by default or selected with -u

---

## Command

the permitted executable and arguments

---

## Policy

rules from sudoers and included files

---

## Audit

logs record the privileged request

# Permissions are three triplets evaluated in order

- **rwX** **r-X** - - -

OWNER

GROUP

OTHERS

**rwX**

**r-X**

- - -

file: read · change · run    directory: list names · create/remove · enter/traverse



# Access checks select one permission class

## Owner match

use only the owner triplet

---

## Else group match

use only the group triplet

---

## Else

use the other triplet

---

## No fallback

failed owner bits do not try group bits

---

## Privilege

root and capabilities can alter normal rules

# Directory r, w, and x control different abilities

## Read · r

- list entry names
- does not reveal metadata alone
- ls may show names only
- needs x for useful lookup

## Write · w

- create entries
- remove entries
- rename entries
- usually requires x too

## Execute · x

- traverse the directory
- look up named entries
- reach deeper paths
- not 'run the directory'

For directories, x means traversal and w changes names in that directory.

# Parent directories participate in every path lookup

|                  |                                      |
|------------------|--------------------------------------|
| <b>/</b>         | traverse root                        |
| <b>/srv</b>      | traverse service data                |
| <b>/srv/team</b> | traverse team directory              |
| <b>file</b>      | apply file read or write bits        |
| <b>namei -l</b>  | display every component and its mode |

# 4 ♦ 2 ♦ 1 makes numeric modes readable

r = 4

w = 2

x = 1

---

640 = rw- r-- ---

● ● ● SAME INTENT, VERIFIED

```
chmod 640 report.txt
chmod u=rw,g=r,o= report.txt
stat -c "%A %a %U:%G %n" report.txt
```

# Symbolic chmod states exactly who and what changes

|           |                                            |
|-----------|--------------------------------------------|
| u         | owner class                                |
| g         | group class                                |
| o         | other class                                |
| ✦ / -     | add or remove selected bits                |
| =         | set the selected class exactly             |
| chmod g✦r | add group read without changing other bits |

# umask removes permissions; it never grants

## REQUESTED MODE

### program default

- Files commonly request 666
- Directories commonly request 777
- Execute is not added to ordinary files

## UMASK

### bits to clear

- 022 yields files 644
- 022 yields directories 755
- Process-local and inherited

**Final mode = requested permissions with masked bits removed.**

# Shared work needs the right group, not world access

TEAM SHARE

```
sudo install -d \  
  -o root -g courseops -m 2770 \  
  /srv/course-share  
  
sudo chown alice:courseops status.txt  
sudo chmod 640 status.txt
```

## 2

### setgid on a directory

New files inherit the directory group, keeping collaboration predictable.

**Owner ♦ group ♦ mode must tell the same story.**

# setgid keeps shared directories in one group

## Directory owner

usually an administrator or service account

---

## Directory group

the collaborating team

---

## Mode 2xxx

setgid bit on the directory

---

## New entries

inherit the directory group

---

## Verification

stat and create a file as a real member



# Sticky protects names in shared directories

## WITHOUT STICKY

### shared write

- Any writer may remove entries
- File ownership does not protect names
- Easy for users to disrupt each other

## WITH STICKY

### mode 1xxx

- Only owner, directory owner, or root removes
- Typical example is /tmp
- Shown as t in the other execute position

**Sticky controls deletion and rename, not file confidentiality.**

# ACLs extend permissions without replacing the base model

## MODE BITS

### three classes

- Owner, group, other
- Visible in `ls -l`
- Portable baseline

## POSIX ACL

### named entries

- Additional users or groups
- Mask limits effective named rights
- Inspect with `getfacl`

**Use ACLs deliberately and document why the exception exists.**

# Nano is enough to make a safe first edit

**^ means Ctrl**

```

NANO

nano ~/linux-course/notes/day1.txt

^O Write Out / save
^X Exit
^W Search
^K Cut line
^U Paste line
    
```

- Open the exact path
- Make one small change
- Save, exit, then verify
- Use sudoedit for protected files

**Editing is not verification.**

# Safe edits use backup, diff, and validation

1

## Inspect

read the current file and  
ownership

2

## Protect

copy or use version  
control where  
appropriate

3

## Edit

nano or sudoedit the  
exact path

4

## Validate

use the program's  
syntax checker

5

## Apply

reload or restart, then  
verify behavior

**Saving a file proves only that bytes changed. It does not prove the configuration is valid or active.**

# Live demo: repair access with the smallest change

● ● ● OBSERVE → EXPLAIN → REPAIR → RETEST

```
id alice
stat -c "%A %a %U:%G %n" /srv/demo-share/report.txt
namei -l /srv/demo-share/report.txt

# observe failure as another identity
# repair owner/group/mode
# repeat the exact read
```

**Never begin with chmod 777.**

# Diagnose permission failures from identity to behavior

1

## Identity

id and groups for the  
failing process

2

## Path

namei -l every  
component

3

## Object

stat owner, group,  
mode, ACL

4

## Policy

sudo, service user,  
mount, or security layer

5

## Retest

repeat as the intended  
non-root identity

**Changing mode before identifying the selected class destroys evidence and often widens access unnecessarily.**

# SSH authenticates both ends of the conversation

## CLIENT

**“Is this really my server?”**

Checks the server host key  
against `known_hosts`



## SERVER

**“Is this user authorized?”**

Checks a signature against  
`authorized_keys`

**Encryption without host verification can still connect you to an attacker.**

# SSH separates transport, host, and user trust

## Transport

encrypts and protects session integrity

---

## Host key

proves which server endpoint answered

---

## User proof

proves which account may log in

---

## Account policy

server maps proof to account policy

---

## Session

starts command, shell, forwarding, or subsystem



# Host keys identify servers; user keys identify people

## HOST KEY

### server identity

- Private half stays on server
- Client stores public fingerprint
- Change requires investigation

## USER KEY

### user identity

- Private half stays with user
- Server stores public key
- Can be revoked per account

**Never copy either private key to the opposite side.**

# Verify host keys through an independent channel

**1****Obtain**

get the expected  
fingerprint from an  
administrator

**2****Connect**

observe the presented  
fingerprint

**3****Compare**

match algorithm and  
complete fingerprint

**4****Accept**

store the verified key in  
known\_hosts

**5****Recheck**

treat later changes as an  
event

**Typing yes without comparison is trust on first use, not verification.**

# The private key proves; the public key permits

## PRIVATE KEY

`~/.ssh/id_ed25519`

- Stays on the client
- Protect with a passphrase
- Never email or copy to server

## PUBLIC KEY

`~/.ssh/id_ed25519.pub`

- May be copied to servers
- Stored in `authorized_keys`
- Can be revoked independently

● ● ● GENERATE ON THE CLIENT

```
ssh-keygen -t ed25519 -a 64 -C "linux-course"
```

# A passphrase and agent reduce private-key exposure

## Phrase

a passphrase encrypts the private key file at rest

---

## KDF rounds

increase cost of password guessing

---

## Agent

holds an unlocked key for a limited session

---

## Permissions

prevent other local users from reading the file

---

## Rotation

replace compromised or obsolete credentials

# OpenSSH checks every component of the authorization path

|                |                                            |
|----------------|--------------------------------------------|
| Home directory | owned by the account; not broadly writable |
|----------------|--------------------------------------------|

---

|        |                   |
|--------|-------------------|
| ~/.ssh | normally mode 700 |
|--------|-------------------|

---

|                 |                   |
|-----------------|-------------------|
| authorized_keys | normally mode 600 |
|-----------------|-------------------|

---

|                 |                                 |
|-----------------|---------------------------------|
| Public key line | complete algorithm and key data |
|-----------------|---------------------------------|

---

|         |                                          |
|---------|------------------------------------------|
| Account | login shell and policy must allow access |
|---------|------------------------------------------|

---

|            |                                     |
|------------|-------------------------------------|
| Server log | explains rejected ownership or mode |
|------------|-------------------------------------|

# Put safe SSH defaults in a named client profile

~/.ssh/config

```
Host linux-course
  HostName 192.168.64.10
  User trainee
  IdentityFile ~/.ssh/linux_course_ed25519
  IdentitiesOnly yes
  StrictHostKeyChecking accept-new
  ServerAliveInterval 60
```

- One alias replaces repeated flags
- IdentitiesOnly avoids key roulette
- Verify the first host fingerprint
- Use “yes” after onboarding for strictness

**Same OpenSSH syntax on macOS and Windows.**

# SSH uses the first value obtained

## Command line

explicit options for this invocation

---

## User config

~/.ssh/config patterns in order

---

## System config

/etc/ssh/ssh\_config defaults

---

## Host patterns

specific entries should appear before broad ones

---

## Effective view

ssh -G alias prints resolved settings

# Live demo: verify before you trust

● ● ● SERVER FINGERPRINT + USER KEY

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ''

ssh-keyscan -p 2222 localhost >> ~/.ssh/known_hosts
ssh-keygen -F '[localhost]:2222'
ssh-copy-id -i ~/.ssh/id_ed25519.pub -p 2222 operator@localhost

ssh -o BatchMode=yes -p 2222 \
  operator@localhost 'id; hostname'
```

**Private key: client only. Public key: server. Host key: verify out of band.**



# Test SSH failures one layer at a time

**1****Network**

resolve name and reach  
the port

**2****Host trust**

inspect `known_hosts`  
and fingerprint

**3****Client choice**

`ssh -G` shows user, key,  
and port

**4****User auth**

`ssh -vv` reveals offered  
and accepted methods

**5****Server policy**

read `ssh` service logs  
and account state

**Verbose output is most useful after the basic target, port, and identity are confirmed.**

# Linux logs live in files and in the systemd journal

## TEXT FILES

### `/var/log`

- Application-specific files
- APT and dpkg history
- Rotation changes filenames

## STRUCTURED JOURNAL

### `journalctl`

- Unit, boot, PID, priority fields
- Time-aware filtering
- May be volatile or persistent

Start with the service and time window—not “all logs”.

# A useful log record answers five questions

**When**

timestamp with timezone or boot context

---

**Where**

host, container, or node

---

**Who**

unit, process, PID, or identity

---

**What**

event and severity

---

**Why next**

correlation field, exit code, or causal detail

---

# Filter the journal toward one answer

Which service? `journalctl -u ssh`

---

Which time? `--since "-10 min"`

---

Which severity? `-p warning`

---

How much? `-n 50 --no-pager`

---

Live? `-f`

---

# journalctl can narrow by unit, boot, time, and priority

|               |                                            |
|---------------|--------------------------------------------|
| Unit          | <code>-u ssh.service</code>                |
| Current boot  | <code>-b</code>                            |
| Previous boot | <code>-b -1</code>                         |
| Time          | <code>--since '-10 minutes'</code>         |
| Priority      | <code>-p warning</code>                    |
| Stable output | <code>-n 50 --no-pager -o short-iso</code> |

# Filter narrowly, then widen one dimension at a time

## Bound

- Choose one unit
- Select one boot
- Set the time window
- Name the incident

## Read

- Follow timestamps
- Find the first cause
- Capture exit reason
- Note dependencies

## Widen

- Extend time
- Add priorities
- Inspect related units
- Inspect the process

**Do not begin with all logs and search by intuition.**

# Live demo: ask the journal a smaller question

● ● ● UNIT → TIME → PRIORITY → FOLLOW

```
sudo systemctl restart ssh
systemctl status ssh --no-pager
```

```
sudo journalctl -u ssh -n 10 --no-pager
sudo journalctl -u ssh --since "-5 min" -p warning --no-pager
sudo journalctl -u ssh -f
```

**Read timestamps and the first causal message aloud.**

# History and live follow serve different jobs

## BOUNDED QUERY

### incident review

- Repeatable output
- Easy to save and compare
- Best for finding first cause

## FOLLOW · -f

### live observation

- Shows new entries
- Useful while reproducing
- Stop with Ctrl+C

Capture a bounded baseline before entering follow mode.

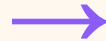


# systemd manages declared units toward a state

## UNIT FILE

### What should run?

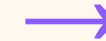
dependencies · user · command  
restart policy · boot target



## MANAGER

### What is running?

start · stop · supervise  
track exit status



## JOURNAL

### What happened?

events + output

“Enabled” means start at boot. “Active” means running now. They are independent.

# Unit files come from layered locations

## Vendor units

/usr/lib/systemd/system

---

## Admin units

/etc/systemd/system and overrides

---

## Runtime units

/run/systemd/system

---

## Drop-ins

name.service.d/\*.conf overrides selected settings

---

## Effective view

systemctl cat name.service

# Service transitions leave journal evidence

## Declared

- ExecStart command
- Service user
- Dependencies
- Restart policy

## Transition

- Conditions
- Ordering
- Credentials
- Exit status

## Observed

- Active state
- Substate
- Main PID
- Journal events

**systemctl status is a current summary; journalctl provides the longer event sequence.**

# Requirement and ordering are separate relationships

|                  |                                        |
|------------------|----------------------------------------|
| <b>Requires=</b> | start together; failure relationship   |
| <b>Wants=</b>    | weaker activation relationship         |
| <b>After=</b>    | ordering only; does not pull a unit in |
| <b>Before=</b>   | inverse ordering relationship          |
| <b>PartOf=</b>   | propagate selected lifecycle actions   |

# Use systemctl verbs deliberately

Observe `status` · `is-active` · `is-enabled`

---

Change runtime `start` · `stop` · `restart` · `reload`

---

Change boot `enable` · `disable`

---

Inspect config `cat` · `show`

---

After unit edits `daemon-reload`

---

**Restart a service only after capturing evidence.**

# Enabled, active, reloaded, and restarted are independent facts

## BOOT + RUNTIME

### enable / start

- enable changes boot links
- start changes current state
- --now requests both operations

## CONFIG APPLY

### reload / restart

- reload asks process to reread
- restart replaces runtime process
- daemon-reload rereads unit files

**Name the state you intend to change before choosing the verb.**

# Live demo: status is the symptom; logs reveal the cause

● ● ● FAILED SERVICE → VERIFIED ENDPOINT

```
systemctl status demo-web --no-pager
sudo journalctl -u demo-web --since "-5 min" --no-pager
systemctl cat demo-web

# repair missing user + working directory
sudo systemctl restart demo-web
systemctl is-active demo-web
curl -fsS http://localhost:8090
```

**State ♦ logs ♦ configuration ♦ behavior**

# Diagnose services from symptom to verified behavior

1

## State

systemctl status and is-active

2

## Events

journalctl by unit, boot, and time

3

## Definition

systemctl cat and show

4

## Repair

change one evidenced cause

5

## Behavior

test the endpoint as a real client

**A green active state proves that systemd sees a running process, not that users receive the intended service.**



# Live demo: separate the first cause from later noise

|            |                                                                |
|------------|----------------------------------------------------------------|
| Status     | <code>systemctl status course-web --no-pager</code>            |
| Events     | <code>journalctl -u course-web -b --since '-10 min'</code>     |
| Definition | <code>systemctl cat course-web</code>                          |
| Repair     | <b>change only the evidenced cause</b>                         |
| Restart    | <b>systemctl restart course-web</b>                            |
| Verify     | <b>is-active plus <code>curl http://127.0.0.1:8088/</code></b> |

# A safe operator closes every change with evidence

1

**Locate**

browser or session ·  
user · directory

2

**Predict**

what state should  
change?

3

**Act**

one scoped command

4

**Observe**

output · status · logs

5

**Verify**

behavior through  
another check

**Pair prompt: explain this loop using the failed-service demo.**

HANDS-ON BLOCK

# Three hours. One browser. Six checkpoints.

Work in pairs when useful. Driver types; navigator predicts and checks.

BROWSER LAB: open your assigned seat link

01

02

03

04

05

06

# Labs 1–2 build the map, then turn data into evidence

## LAB 1 • 25 MIN

### Filesystem fieldwork

```
ORIENT • BUILD • VERIFY

p=~/projects/linux-course; pwd; whoami
mkdir -p "$p"/{notes,archive}
cp ~/inbox/welcome.txt "$p"/notes/
mv ~/inbox/draft.txt "$p"/archive/
```

## LAB 2 • 25 MIN

### Packages ♦ pipelines

- Install tree through APT
- Inspect package state
- Count HTTP status codes
- Write a verified report

**Checkpoint:** the project tree is correct and 404 appears exactly three times.

# Labs 3–4 turn access rules into verified trust

## LAB 3 · 35 MIN

### Permission incident

● ● ● OWNER · GROUP · MODE

```
sudo chown -R root:webteam /srv/team-  
share  
sudo chmod 2770 /srv/team-share  
sudo -u alice test -w /srv/team-share  
sudo -u outsider test -w /srv/team-  
share || true
```

## LAB 4 · 30 MIN

### SSH trust setup

- Generate an Ed25519 key
- Record the server host key
- Authorize only the public key
- Connect through a safe alias

**Checkpoint: Alice writes, outsider is blocked, and key-only SSH succeeds.**

# Labs 5–6 turn evidence into a durable repair

## LAB 5 • 30 MIN

### Journal investigation

- Start with service status
- Filter by unit + boot
- Find the first causal event
- Record incident + exit code

## LAB 6 • 35 MIN

### Broken service capstone

EVIDENCE → REPAIR

```
systemctl status course-web --no-pager
journalctl -u course-web -b --no-pager
systemctl cat course-web
sudo systemctl enable --now course-web
curl -fsS http://127.0.0.1:8088/
```

**Final checkpoint: active ♦ enabled ♦ HTTP response.**

# When blocked, widen evidence before widening privilege

- 1 Context** `hostname · whoami · pwd`
- 2 Target** `ls -ld · stat · systemctl cat`
- 3 Symptom** `repeat the exact failing action`
- 4 Evidence** `status · journalctl · exit status`
- 5 Change** `one cause; then retest`

# The final challenge is to explain, not merely succeed

## Tell your partner:

- Which machine and identity ran the command?
- What evidence identified the service failure?
- Why did the permission repair preserve least privilege?
- How did you verify SSH server identity?
- What proved the web service behavior?

---

**If you can explain the state, you can recreate the command.**

---



# Keep your seat link; keep practising the loop

## NEXT 24 HOURS

- Reset Lab 6 and solve without hints
- Rebuild the SSH profile from memory
- Explain 640 and 2770 aloud

## NEXT 30 DAYS

- Continue with Linux Upskill Challenge
- Read one man page per session
- Automate only after the manual path is clear

**Locate → predict → act → observe → verify**

# Sources and open-course attribution

## OPEN COURSE INSPIRATION

Linux Upskill Challenge · CC BY 4.0  
linuxupskillchallenge.com

Linux Training + Linux101Labs  
coverage and exercise-pattern review

## PRIMARY REFERENCES

Canonical Ubuntu Server documentation  
Incus system-container documentation  
ttyd browser-terminal project  
OpenSSH + systemd manuals

---

## Visual direction

Platform source and delivery notes: [browser-lab-platform/README.md](#)

**Full URLs, licensing notes, and adaptation details: [resources/SOURCES.md](#)**